

C++ Tokens

C++ Tokens

- C++ is the superset of C and so most constructs of C are legal in C++ with their meaning and usage unchanged. So tokens, expressions, and data types are similar to that of C. However, there are some exceptions and additions in C++ which you will not get in C.

C++ Tokens

- Each word and punctuation is referred to as a token in C++. Tokens are the smallest building block or smallest unit of a C++ program.
- These following tokens are available in C++:
 - Keywords
 - Identifiers
 - Constants
 - Strings
 - Special Symbols
 - Operators

Keywords

- Keywords are reserved words which have fixed meaning, and its meaning cannot be changed. The meaning and working of these keywords are already known to the compiler. C++ has more numbers of keyword than C, and those extra ones have special working capabilities.
- C language supports 32 keywords which are also supported by C++, which are given below:

Keywords in C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Keyword in C++

- While in C++ there are 31 additional keywords other than C Keywords they are:

asm	bool	catch	class
const_cast	delete	dynamic_cast	explicit
export	false	friend	inline
mutable	namespace	new	operator
private	protected	public	reinterpret_cast
static_cast	template	this	throw
true	try	typeid	typename
using	virtual	wchar_t	

Identifiers

- Identifiers are names given to different entries such as variables, structures, and functions. Also, identifier names should have to be unique because these entities are used in the execution of the program.
- Rules for identifiers:
 - Names can contain letters, digits and underscores
 - Names must begin with a letter or an underscore (_)
 - Names are case sensitive (myVar and myvar are different variables)
 - Names cannot contain whitespaces or special characters like !, #, %, etc.
 - Reserved words (like C++ keywords, such as int) cannot be used as names
 - The major difference between C and C++ is the limit on the length of the name of the variable. ANSI C considers only the first 32 characters in a name while ANSI C++ imposes no limit on the length of the name.

Constants

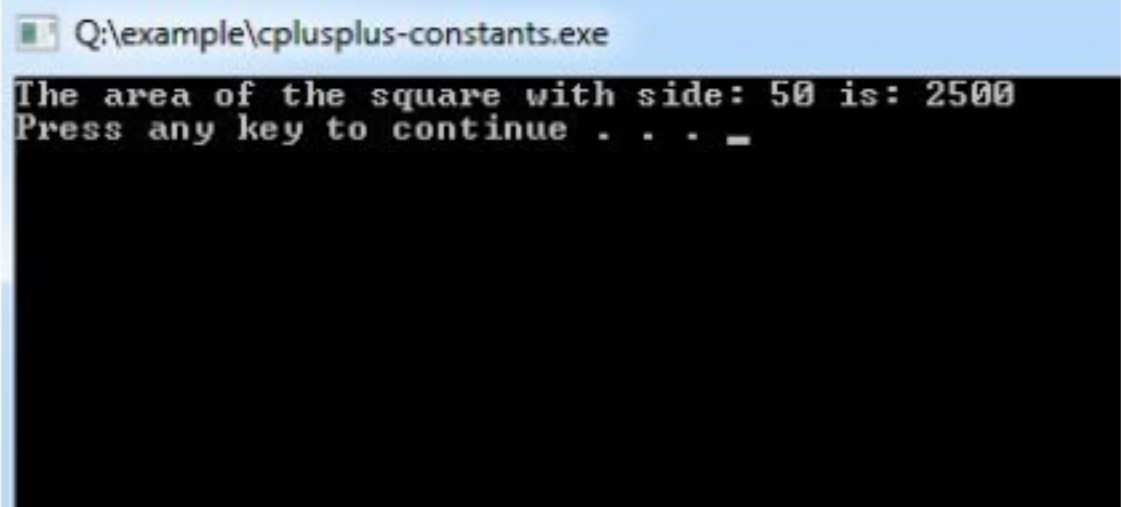
- Constants are also like normal variables. But, the only difference is, their values can not be modified by the program once they are defined. Constants refer to fixed values. They are also called **literals**.
- Constants may belong to any of the data type.
- There are two other different ways to define constants in C++. These are:
 - By using **const** keyword
 - By using **#define** preprocessor

Constant Defination by using **const** Keyword

- Syntax: **const** type constant_name=value;

```
#include <iostream>
using namespace std;
```

```
int main()
{
    const int SIDE = 50;
    int area;
    area = SIDE*SIDE;
    cout<<"The area of the square with side: " << SIDE <<" is: "
    << area << endl;
    return 0;
}
```



The screenshot shows a Windows command prompt window titled "Q:\example\cplusplus-constants.exe". The output text is: "The area of the square with side: 50 is: 2500" followed by "Press any key to continue . . . _".

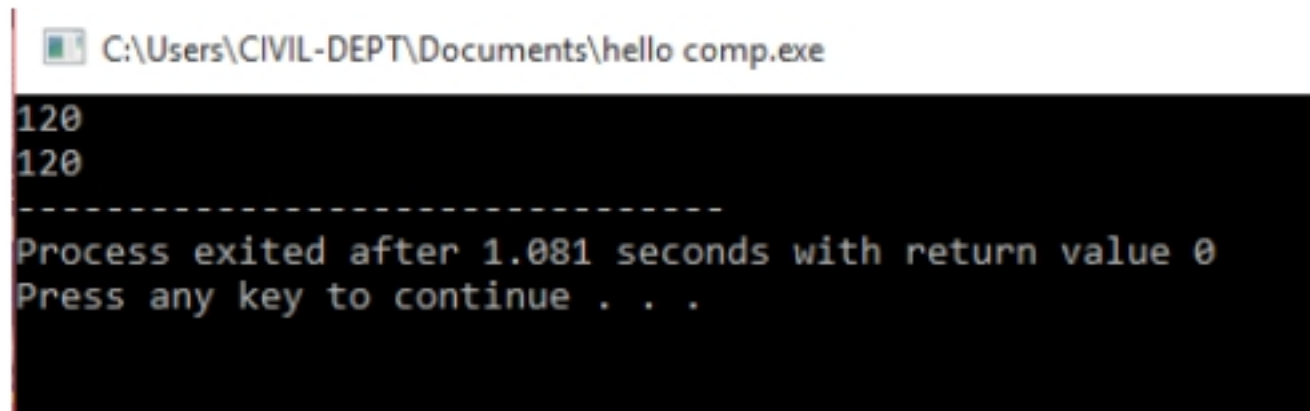
Constant Defination by Usinng #define Preprocessor

Syntax: #define constant_name value;

```
#include <iostream>
using namespace std;
```

```
#define VAL1 20
#define VAL2 6
#define Newline '\n'
```

```
int main()
{
    int tot;
    tot = VAL1 * VAL2;
    cout << tot;
    cout << Newline;
    cout<< tot;
    return 0;
}
```



```
C:\Users\CIVIL-DEPT\Documents\hello comp.exe
120
120
-----
Process exited after 1.081 seconds with return value 0
Press any key to continue . . .
```

String

- The C-style character string originated within the C language and continues to be supported within C++. This string is actually a one-dimensional array of characters which is terminated by a null character '\0'. Thus a null-terminated string contains the characters that comprise the string followed by a null.
- Example:
 - `char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};`
- If you follow the rule of array initialization, then you can write the above statement as follows:
 - `char greeting[] = "Hello";`

Special Symbols

- Apart from letters and digits, there are some special characters in C++ which help you manipulate or perform data operations. Each special symbol has a specific meaning to the C++ compiler. Some of the special Symbol characters:
- [] Square brackets
- () Simple brackets
- { } Curly braces
- , Comma
- # Hash / Pound / Preprocessor
- * Asterisk
- ~ Tilde
- . Period / dot

Operators

- Operators are tools or symbols which are used to perform a specific operation on data. Operations are performed on operands. Operators can be classified into three broad categories according to the number of operands used.
- **Unary:** It involves the use of one a single operand. Like ! , ++, --, -. +
- **Binary:** It involves the use of 2 operands. They are further classified as:
 - Arithmetic
 - Relational
 - Logical
 - Assignment
 - Bitwise
- Ternary: conditional: It involves the use of 3 operands. For instance, ?: is used to in place of if-else conditions.

New operators in c++

- C++ introduces some new operators given as below:
- | Symbols | Name |
|---------|-------------------------------------|
| << | Insertion operator. |
| >> | Extraction operator. |
| :: | Scope resolution operator. |
| ::* | Pointer-to-member declarator. |
| ->* | Pointer-to-member operator. |
| .* | Pointer-to-member operator. |
| new | Dynamic memory allocation operator. |
| delete | Memory release operator. |

new and delete Operators

- **new** is used to allocate memory for a variable, object, array, array of objects.. etc at run time. "Declaring memory at run time is known as dynamic memory allocation." we use memory by this method (dynamic allocation) when it is not known in advance how much memory space is needed.
- The new operator can be used to create objects of any type, it takes the following general form:
- Syntax:
 - `pointer_variable = new data_type;`
 - `pointer_variable = new data_type[Size];`
 - Here, `pointer_variable` is a variable that store the starting address of allocating memory space.

Example of new Operator

```
#include <iostream.h>
int main(){
    int *age;
    age=new int;
    char *gender=new char;
    *age=0;
    *gender='M';
    cout<<"Enter your age ? :";
    cin>> (*age);
    cout<<"AGE IS : "<< *age << ",GENDER IS : "<< *gender << endl;
    delete age;
    delete gender;
    return 0;
}
```

```
Enter your age ? :23
AGE IS : 23,GENDER IS : M
```

delete Operator

- **delete** operator is used to Deallocates the dynamically allocated memory.
- Since the necessity of dynamic memory is usually limited to specific moments within a program, once it is no longer needed it should be freed so that the memory becomes available again for other requests of dynamic memory. This is the purpose of the operator delete.
- delete takes the following general form:
 - delete pointer_variable;
 - delete []pointer_variable;
- The first expression should be used to delete memory allocated for a single element, and the second one for memory allocated for arrays of elements.

Example of delete Operator

```
#include <iostream.h>
int main(){
    int *array;
    int max,i;

    cout<<"Enter total number of elements :";
    cin>>max;

    array=new int[max];
    cout<<"Enter array elements:\n";
    for(i=0;i< max;i++){
        cout<<"Element "<< i+1 <<": ";
        cin>> array[i];
        // cin>>*(array+i);
    }
    cout<<"Elements are : ";
    for(i=0;i< max;i++){
        cout<< array[i] <<"\t";
    }
    delete []array;
    cout<< endl;
    return 0;
}
```

```
Enter total number of elements :5
Enter array elements:
Element 1: 111
Element 2: 222
Element 3: 333
Element 4: 444
Element 5: 555
Elements are : 111      222      333      444      555
```

Data Types

- All variables use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data-type with which it is declared. Every data type requires a different amount of memory.

Data types in C++

```
graph TD; A[Data types in C++] --> B[User Defined]; A --> C[Built-in]; A --> D[Derived]; B --> E["Structure, Union, Enum"]; C --> F["int, char, float, double, boolean, void"]; D --> G["Array, Function, Pointer"];
```

User Defined

Structure,
Union,
Enum

Built-in

int, char,
float, double
boolean,
void

Derived

Array,
Function,
Pointer

Data types in C++ is mainly divided into three types:

- **Primitive Data Types:** These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char , float, bool etc. Primitive data types available in C++ are:
 - Integer
 - Character
 - Boolean
 - Floating Point
 - Double Floating Point
 - Valueless or Void
- **Derived Data Types:** The data-types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:
 - Function
 - Array
 - Pointer
 - Reference

Data types in C++ is mainly divided into three types:

- **Abstract or User-Defined Data Types:** These data types are defined by user itself. Like, defining a class in C++ or a structure. C++ provides the following user-defined datatypes:
 - Class
 - Structure
 - Union
 - Enumeration
 - Typedef defined DataType

Enumeration type in C++

- Enum is a user defined data type where we specify a set of values for a variable and the variable can only take one out of a small set of possible values. We use enum keyword to define a Enumeration.
- **enum** direction {East, West, North, South} dir;
- Here Enumeration name is direction which can only take one of the four specified values, the dir at the end of the declaration is an enum variable.

Example:

```
#include <iostream>
using namespace std;
enum direction {East, West, North, South};
int main()
{
    direction dir;
    dir = South;
    cout<<dir;
    return 0;
}
```

Here I have assigned the value South to the enum variable dir and when I displayed the value of dir it shows 3. This is because by default the values are in increasing order starting from 0, which means East is 0, West is 1, North is 2 and South is 3.

Example

```
#include <iostream>
using namespace std;
enum direction {East=11, West=22, North=33, South=44};
int main()
{
    direction dir;
    dir = South;
    cout<<dir;
    return 0;
}
```

Output:

44

```
#include <iostream>
using namespace std;
```

```
enum week { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday,
Saturday };
```

```
int main()
{
    week today;
    today = Wednesday;
    cout << "Day " << today+1;
    return 0;
}
```

Output:
Day 4

```
#include<iostream>
using namespace std;
enum year { Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
int main()
{
    int i;
    for (i = Jan; i <= Dec; i++)
        cout << i +1<< "\t";
    return 0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10 11 12

Type Conversion

- Type Conversion refers to conversion from one type to another. The main idea behind type conversion is to make variable of one type compatible with variable of another type to perform an operation. For example, to find the sum of two variables, one of int type & other of float type. So, you need to type cast int variable to float to make them both float type for finding the sum.
- In C++, there are two types of type conversion.
 - Implicit Conversion
 - Explicit Conversion (also known as Type Casting)

Implicit Type Conversion

- Implicit type conversion or automatic type conversion is done by the compiler on its own. There is no external trigger required by the user to typecast a variable from one type to another.
- This occurs when an expression contains variables of more than one type. So, in those scenarios automatic type conversion takes place to avoid loss of data. In automatic type conversion, all the data types present in the expression are converted to data type of the variable with the largest data type.
- Below is the order of the automatic type conversion. You can also say, smallest to largest data type for type conversion.
 - bool -> char -> short int -> int -> unsigned int -> long -> unsigned long -> float -> double -> long double
- Implicit conversions can lose information such as signs can be lost when signed type is implicitly converted to unsigned type and overflow can occur when long is implicitly converted to float.

Higher Data Type

long double

double

float

long

int

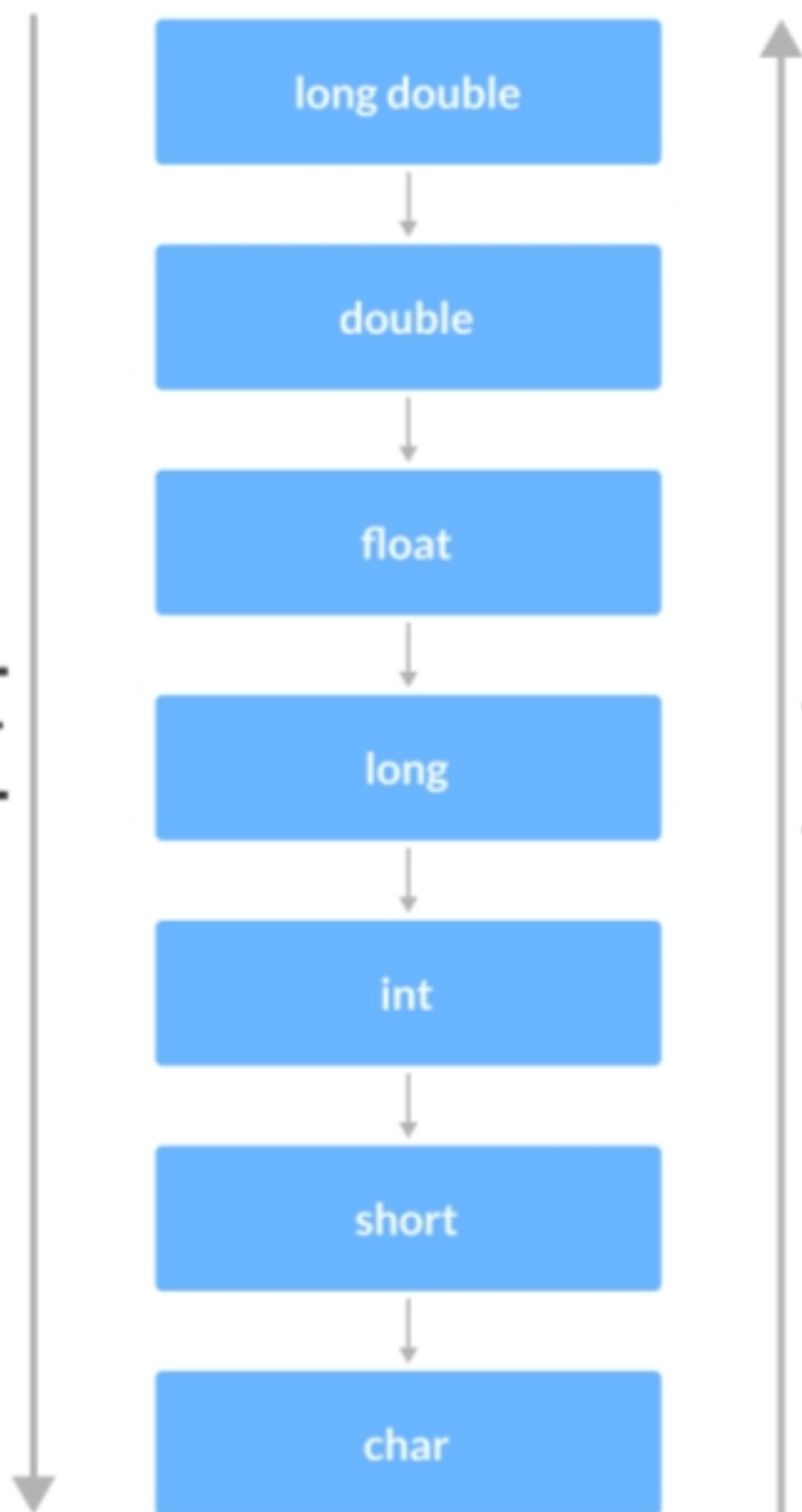
short

char

Lower Data Type

data loss

no data loss



Example

```
#include <iostream>
using namespace std;
int main()
{
    int int1 = 100;
    char char1 = 'c';
    // char1 implicitly converted to int using ASCII value of 'c' i.e. 99
    int1 = int1 + char1;
    // int1 is implicitly converted to float
    float flt1 = int1 + 2.7;
    cout << "int1 = " << int1 << endl
    << "char1 = " << char1 << endl
    << "flt1 = " << flt1 << endl;
    return 0;
}
```

Output:

int1 = 199

char1 = c

flt1 = 201.7

Explicit Type Conversion

- When the user manually changes data from one type to another, this is known as explicit type conversion. This type of conversion is also known as type casting.
- There are two major ways in which we can use explicit conversion in C++. They are:
 - C-style type casting (also known as cast notation)
 - Function notation (also known as old C++ style type casting)

C-style type casting (also known as cast notation)

- As the name suggests, this type of casting is favored by the C programming language. It is also known as cast notation.
- **syntax:**
 - (data_type)expression;
- **Example:**
 - int num_int = 26;
 - double num_double;
- converting from **int** to **double**
 - num_double = (double)num_int;

Function-style Casting

- We can also use the function like notation to cast data from one type to another. also known as old C++ style type casting
- **Syntax :**
 - `data_type(expression);`
- **Example:**
 - `int num_int = 26;`
 - `double num_double;`
- converting from **int** to **double**
- `num_double = double(num_int);`

Example

```
#include <iostream>

using namespace std;

int main() {
    double num_double = 3.56;
    cout << "num_double = " << num_double << endl;

    // C-style conversion from double to int
    int num_int1 = (int)num_double;
    cout << "num_int1  = " << num_int1 << endl;

    // function-style conversion from double to int
    int num_int2 = int(num_double);
    cout << "num_int2  = " << num_int2 << endl;

    return 0;
}
```

Output:
num_double = 3.56
num_int1 = 3
num_int2 = 3

Manipulators

- Manipulators are helping functions that can modify the input/output stream available in `iomanip.h` header file. It does not mean that we change the value of a variable, it only modifies the I/O stream using insertion (<<) and extraction (>>) operators.
- Some important manipulators:
 - `setw()`
 - `setfill()`
 - `setf()//setiosflag()`
 - `setprecision()`

Setw() and Setfill() manipulators

- **setw** manipulator sets the width of the field assigned for the output. The field width determines the minimum number of characters to be written in some output representations. If the standard width of the representation is shorter than the field width, the representation is padded with fill characters (using setfill).
- **setfill** character is used in output insertion operations to fill spaces when results have to be padded to the field width.
- **Syntax**
 - `setw([number_of_characters]);`
 - `setfill([character]);`

Example

```
#include <iostream.h>
#include <iomanip.h>
int main()
{
    cout<<"USING setw() .....\\n";
    cout<< setw(10) <<11<<"\\n";
    cout<< setw(10) <<2222<<"\\n";
    cout<< setw(10) <<33333<<"\\n";
    cout<< setw(10) <<4<<"\\n";

    cout<<"USING setw() & setfill() [type- I]...\\n";
    cout<< setfill('0');
    cout<< setw(10) <<11<<"\\n";
    cout<< setw(10) <<2222<<"\\n";
    cout<< setw(10) <<33333<<"\\n";
    cout<< setw(10) <<4<<"\\n";

    cout<<"USING setw() & setfill() [type-II]...\\n";
    cout<< setfill('-')<< setw(10) <<11<<"\\n";
    cout<< setfill('*')<< setw(10) <<2222<<"\\n";
    cout<< setfill('@')<< setw(10) <<33333<<"\\n";
    cout<< setfill('#')<< setw(10) <<4<<"\\n";
    return 0;
}
```

OUTPUT

```
USING setw() .....  
    11  
   2222  
  33333  
   4  
USING setw() & setfill() [type- I]...  
0000000011  
0000002222  
0000033333  
0000000004  
USING setw() & setfill() [type-II]...  
-----11  
*****2222  
@@@@@33333  
#####4
```

setprecision() manipulator

- setprecision manipulator sets the total number of digits to be displayed, when floating point numbers are printed.
- Syntax
 - `setprecision([number_of_digits]);`
 - `cout<<setprecision(5)<<1234.537;`

DECLARATION OF VARIABLES

- In ANSI C all the variable which is to be used in programs must be declared at the beginning of the program .But in C++ we can declare the variables any where in the program where it requires .This makes the program much easier to write and reduces the errors that may be caused by having to scan back. It also makes the program easier to understand because the variables are declared in the context of their use.

Example:

```
int main( )
{
    float x;
    float sum=0;
    for(int i=1;i<5;i++)
    {
        cin>>x;
        sum=sum+x ;
    }
    float average;
    average=sum/5;
    cout<<average;
    return 0;
}
```

REFERENCE VARIABLES

- C++Introduces a new kind of variable known as the reference variable. A references variable provides an alias.(alternative name) for a previously defined variable. For example ,if we make the variable sum a reference to the variable total, then sum and total can be used interchangeably to represent the variable.
 - Synatx: datatype & reference_name=variable_name;
 - Example: **float** total=1500;
float &sum=total;
- Here sum is the alternative name for variables total, both the variables refer to the same data in the memory.
- A reference variable must be initialized at the time of declaration . Note that C++ assigns additional meaning to the symbol & here & is not an address operator .The notation float & means reference to float.

Example of reference variable

```
#include<iostream>
using namespace std;
void swap (int &num1, int &num2)
{
    int temp;
    temp=num1;
    num1=num2;
    num2=temp;
}
int main()
{
    int a=5,b=10;
    cout<<"\n Before swapping"<<"\n A = "<<a<<"\n B = "<<b<<endl;
    swap(a,b);
    cout<<"\n After swapping"<<"\n A = "<<a<<"\n B = "<<b<<endl;
    return 0;
}
```

Before swapping

A = 5

B = 10

After swapping

A = 10

B = 5